

Managed application platform or custom infrastructure?

Portail Nova · Startup · Level 1

24 employees · 2 engineers Current step: portfolio 90/100 · needs_revision

 This is a synthetic consulting case created for cloud architecture practice. No real customer data is represented.

01 / DISCOVERY

Discovery

Business brief

A two-person startup must launch a customer portal without spending its entire sprint on infrastructure.

What the client initially shared

- The client needs a justified cloud architecture decision and a feasible transition plan.
- Interview the client to establish workload, service targets, budget and operational constraints.

Stakeholders

Alex Morgan
Business sponsor

Customer impact and business priorities

Priya Shah
CTO

Feasibility, reliability, and engineering capacity

Sam Chen
Lead engineer

How the current system behaves and who operates it

Skip to content

Questions and answers

My question

How much engineering time can the two-person team spend on infrastructure each week before launch, and when must the portal go live?

Alex Morgan

2 engineers maintain the product. Confirm who owns deployment, patching, incidents and recovery before committing to an option. Two engineers; four-week launch; modest traffic; encrypted customer data; \$350/month planning target. (Source: Engineering manager)

A standard Python web app uses PostgreSQL, sends email and runs a nightly report. (Source: Lead engineer)

My question

How many hours per week can the two engineers spend on infrastructure, deployment, patching, and incidents?

Alex Morgan

Two engineers; four-week launch; modest traffic; encrypted customer data; \$350/month planning target. These are planning constraints; compare actual estimates including operations, data transfer and resilience. No supplier quote has been approved. (Source: Budget owner)

A standard Python web app uses PostgreSQL, sends email and runs a nightly report. (Source: Lead engineer)

My question

Provide a monthly cost estimate for each option, including compute, database, data transfer, backups, monitoring, and operational effort.

Alex Morgan

[Skip to content](#)

Two engineers; four-week launch; modest traffic; encrypted customer data; \$350/month planning target. These are planning constraints; compare actual estimates including operations, data transfer and resilience. No supplier quote has been approved. (Source: Budget owner)

My question

Does the application need any custom networking, operating system access, or special server configuration?

Alex Morgan

A standard Python web app uses PostgreSQL, sends email and runs a nightly report. (Source: Lead engineer)

My question

How quickly must the team be able to deploy a new version of the portal, and how often do they expect to deploy?

Alex Morgan

2 engineers maintain the product. Confirm who owns deployment, patching, incidents and recovery before committing to an option. Two engineers; four-week launch; modest traffic; encrypted customer data; \$350/month planning target. (Source: Engineering manager)

My question

Is the \$350 per month target a hard limit, or can the budget go higher if the managed option saves engineering time?

Alex Morgan

Two engineers; four-week launch; modest traffic; encrypted customer data; \$350/month planning target. These are planning constraints; compare actual estimates including operations, data transfer and resilience. No supplier quote has been approved. (Source: Budget owner)

My question

How many users do you expect at launch, and what peak number of concurrent users should the portal support?

I don't have a reliable answer to that yet. Could you narrow the question to a business constraint or a specific measurement? If we aren't measuring it, we can discuss what evidence you need.

My question

What is the maximum number of requests per minute the portal should handle at launch?

Alex Morgan

I don't have a reliable answer to that yet. Could you narrow the question to a business constraint or a specific measurement? If we aren't measuring it, we can discuss what evidence you need.

Evidence requests

No evidence requests recorded yet.

Facts, assumptions, and unknowns

Type	Note	Source / confidence	Validation
Fact	The client needs a justified cloud architecture decision and a feasible transition plan.	Client brief · high	—
Fact	Interview the client to establish workload, service targets, budget and operational constraints.	Client brief · high	—
Fact	Two engineers maintain the product.	lead engineer · medium	—
Fact	The application is a standard Python web app using PostgreSQL, email, and a nightly report	lead engineer · medium	—
Unknown	We do not know who owns deployments, patching, incidents, and recovery.	lead engineer · medium	Confirm operational ownership with the engineering manager before choosing the platform.
Fact	No supplier quote has been approved.	lead engineer · medium	—

Stated Problem	A two-person startup must launch a customer portal without spending its entire sprint on infrastructure.
Actual Problem	The startup needs to launch a secure customer portal in four weeks without spending too much engineering time on infrastructure.
Business Impact	A poor platform choice could delay the launch, increase monthly cost, or create too much operational work for a two-person team.
Symptoms	The application is a standard Python web app using PostgreSQL, email, and a nightly report. Expected traffic is modest, but the exact peak load is not yet known.
Constraints	Two engineers maintain the product. The portal must launch within four weeks. Customer data must be encrypted. The planning target is about \$350 per month.
Uncertainties	We do not know the real peak traffic, the final monthly cost of each option, or how much infrastructure work the two engineers can support. These points could change the platform choice.

03 / REQUIREMENTS

Business and technical requirements

ID	Requirement	Priority	Source
R01	The portal must launch within four weeks.	must	lead engineer
R02	Customer data must be encrypted.	must	lead engineer
R03	Keep the solution around the \$350/month planning target.	must	lead engineer
R04	Cost comparison must include operations, data transfer, and resilience.	must	lead engineer

04 / ARCHITECTURE OPTIONS

Architecture options

[Skip to content](#)

Elastic Beanstalk with managed PostgreSQL

Summary	Deploy the Python application on a managed application platform and use a managed PostgreSQL database. Let AWS handle most of the server setup, health monitoring, and scaling
Components	Elastic Beanstalk runs the Python web application. RDS PostgreSQL stores customer data. Email is sent through a managed email service. A scheduled job runs the nightly report. Encryption is enabled for stored data and network traffic.
Advantages	Fast to deploy within four weeks. Less infrastructure work for the two engineers. Good fit for a standard Python application and modest traffic. Managed database reduces maintenance.
Disadvantages	Less infrastructure control. Some platform behavior is managed by AWS. May become limiting if the application later needs special infrastructure.
Cost	Likely easier to keep near the \$350/month target for modest traffic, but the real monthly estimate still needs validation.
Operations	Low. The team mainly manages the application instead of the underlying infrastructure.
Reliability	Managed health checks, application instances, backups, and database features can improve reliability with limited team effort.
Security	Customer data can be encrypted in transit and at rest. Access permissions still need to be configured correctly.
Implementation	Low. Good fit for a four-week launch.
Assumptions	The application does not require special networking, operating system access, or custom infrastructure.
Risks	Future requirements may outgrow the managed platform. The final monthly cost is not yet validated.
Skip to content	

Tradeoff Rationale

Elastic Beanstalk is a strong fit for the four-week launch and two-engineer team because it reduces infrastructure work. It should be enough for a standard Python application with modest traffic. The trade-off is less control and flexibility if the application becomes more complex later.

OPTION 2

ECS Fargate + RDS

Summary	Package the Python application as a container and run it on ECS Fargate with a managed PostgreSQL database.
Components	ECS Fargate runs the web application containers. A load balancer sends traffic to the containers. RDS PostgreSQL stores customer data. A scheduled container runs the nightly report. Email uses a managed email service.
Advantages	More control over the application runtime. Easier to move or extend the container architecture later. Can scale as traffic grows. No EC2 servers to patch.
Disadvantages	More setup than the managed application platform. Requires container configuration, networking, load balancing, monitoring, and deployment setup. More operational work for a two-person team.
Cost	Can remain reasonable for modest traffic, but the load balancer, containers, database, monitoring, and resilience may make it harder to stay near \$350/month.
Operations	Medium. AWS manages the servers, but the team still manages containers, networking, scaling, and deployments.
Skip to content	Containers can run across multiple zones and restart after failures, but the team must configure this correctly.

Security	Supports encryption and strong network isolation, but there are more permissions and network settings to manage.
Implementation	Medium. Achievable, but it uses more of the four-week launch window.
Assumptions	The team is comfortable with containers and can maintain the additional infrastructure.
Risks	The team may spend too much of the sprint on infrastructure. Monthly cost and operational effort may exceed the startup's planning target.
Tradeoff Rationale	ECS Fargate gives more control, scalability and flexibility for future growth. However, it adds container, networking and deployment work that may not be necessary for a two-engineer startup with a four-week launch deadline and modest traffic.

05 / TRADE-OFFS

Trade-off matrix

Dimension	Elastic Beanstalk with managed PostgreSQL	ECS Fargate + RDS
Cost	4/5	3/5
Reliability	4/5	4/5
Performance	4/5	4/5
Scalability	4/5	5/5
Security	4/5	4/5
Operational Burden	5/5	3/5
Implementation Speed	5/5	3/5
Team Fit	5/5	3/5
Flexibility	3/5	5/5
Reversibility	3/5	4/5

Contextual ratings by the learner · 1–5, higher is more favorable · not measured benchmarks.

[Skip to content](#)

Requirement traceability

Requirement	Decision	Status	Rationale
R01	Use Elastic Beanstalk to deploy the Python application and reduce infrastructure setup.	Satisfied	Elastic Beanstalk reduces the amount of infrastructure the two-engineer team must build and manage, making the four-week launch more realistic.
R02	Encrypt customer data at rest in the managed PostgreSQL database and use HTTPS for data in transit.	Satisfied	Encryption at rest protects stored customer data, while HTTPS protects data between customers and the application.
R03	Use a simple managed application platform and right-sized database resources for the expected modest traffic.	At risk	The architecture is designed to avoid unnecessary infrastructure, but the real monthly cost has not been validated yet.
R04	Build a full monthly cost estimate including application hosting, database, backups, monitoring, data transfer, resilience, and engineering effort.	At risk	These costs are part of the decision, but actual estimates still need to be completed and reviewed before production approval.

06 / RECOMMENDATION

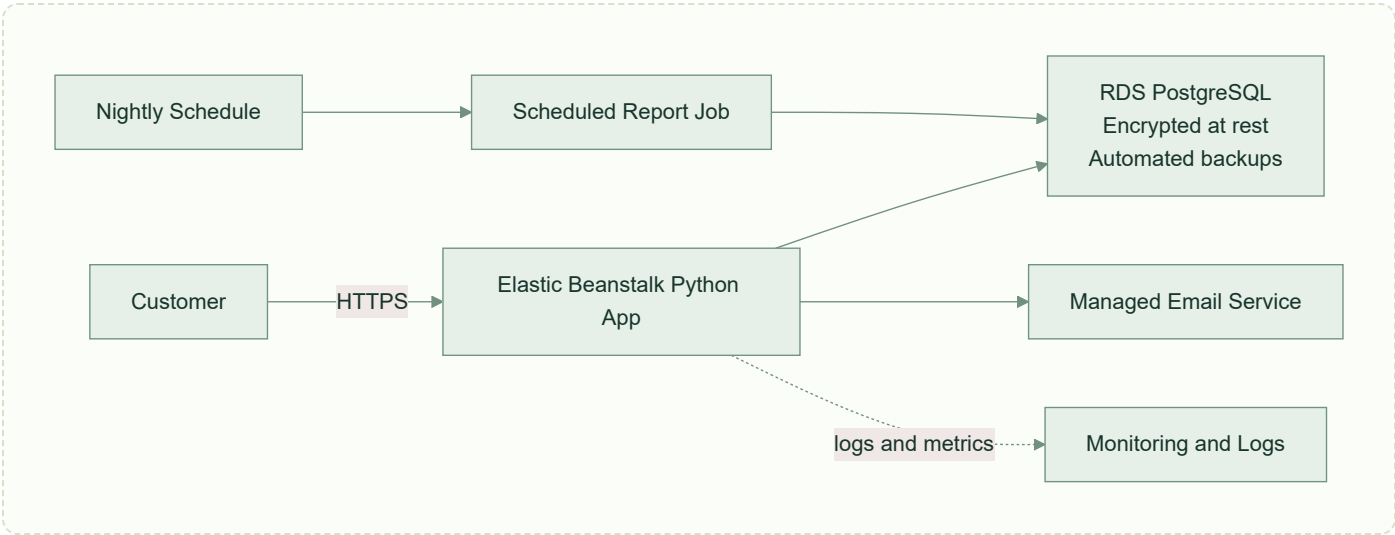
Recommendation

Selected Option	Elastic Beanstalk with managed PostgreSQL
Option Id	dab6959f-96f2-4712-92a4-cdec598dc3ad
Decision	Deploy the Python web application on Elastic Beanstalk and use RDS PostgreSQL for the database. Use managed services for email, backups, monitoring, and encryption
Why	The team has only two engineers and must launch in four weeks. The application is standard, traffic is modest, and there is no confirmed need for advanced container infrastructure. Elastic Beanstalk reduces infrastructure work and lets the team focus on the product.
Alternatives	ECS Fargate gives more control and flexibility, but it adds container, networking, deployment, and scaling work. For this small team and short launch timeline, that extra complexity is not yet justified.
Risks	Elastic Beanstalk gives less infrastructure control and may become limiting if the application becomes more complex. The real monthly cost also needs to be validated against the \$350 planning target.
Skip to content	

Assumptions

We assume the application does not need special operating system access or custom networking, and that expected traffic remains modest.

Architecture diagram



07 / SIMPLER ALTERNATIVE

Why not something simpler?

A single server would be simpler and cheaper, but it would create more maintenance and resilience risk. The managed platform adds some cost, but it better fits the small team, four-week launch, security, and backup requirements.

08 / CHANGE CONDITIONS

What would change my mind?

01	Condition	The managed platform cannot meet the cost target or the application later needs more infrastructure control.
	Signal	Monthly cost, engineering time spent on operations, and new technical requirements.
	Threshold	Monthly cost stays well above the \$350 target, or the team needs custom networking, operating system access, or platform features that the managed option cannot support.
	Alternative	Move to ECS Fargate or another more flexible custom infrastructure option.

09 / VALIDATION

Validation and proof of concept

Required	Yes
Justification	Not recorded.
Hypothesis	Elastic Beanstalk can support the standard Python portal, meet the four-week launch target, keep customer data secure, and stay close to the \$350 monthly planning target.
Scope	Test application deployment, PostgreSQL connection, HTTPS, database encryption, email, nightly reports, backups, monitoring, and basic load handling.
Metrics	Deployment time, application availability, response time, monthly cost, backup success, errors, and engineering hours spent on operations.
Success	The portal launches within four weeks, customer data is encrypted, the application works correctly under expected traffic, backups succeed, and monthly cost stays close to \$350.
Exit	Stop or review the decision if the platform delays the launch, cannot support required features, creates too much operational work, or costs significantly more than the planning target.
Go	Skip to content
	Go if the managed platform meets the launch, security, cost, and operational requirements. No-Go if a key requirement cannot be met, then review ECS Fargate or another option.

ADR working draft

Title	Managed platform for customer portal
Context	A two-person startup must launch a standard Python customer portal within four weeks. The application uses PostgreSQL, email, and a nightly report. Customer data must be encrypted, traffic is expected to be modest, and the planning target is about \$350 per month.
Decision	Use Elastic Beanstalk for the Python application and RDS PostgreSQL for the database. Use HTTPS, encryption at rest, automated backups, monitoring, and managed email services.
Alternatives	ECS Fargate was considered because it provides more control and flexibility. It was not selected because it would add container, networking, deployment, and scaling work that is not justified for the current team size, launch deadline, and modest traffic.
Consequences	The team can focus more on the product and launch faster with less infrastructure work. The trade-off is less control over the underlying platform and possible migration work if future requirements become more complex.
Risks	The main risks are that the final monthly cost may exceed the \$350 planning target, the managed platform may become limiting as the product grows, and some operational ownership is still not fully confirmed.

Accepted decision records

ADR-001 · Managed platform for customer portal	
Context	A two-person startup must launch a standard Python customer portal within four weeks. The application uses PostgreSQL, email, and a nightly report. Customer data must be encrypted, traffic is expected to be modest, and the planning target is about \$350 per month.
Decision	Use Elastic Beanstalk for the Python application and RDS PostgreSQL for the database.
Skip to content	Use HTTPS, encryption at rest, automated backups, monitoring, and managed email services.

Alternatives	ECS Fargate was considered because it provides more control and flexibility. It was not selected because it would add container, networking, deployment, and scaling work that is not justified for the current team size, launch deadline, and modest traffic.
Consequences	The team can focus more on the product and launch faster with less infrastructure work. The trade-off is less control over the underlying platform and possible migration work if future requirements become more complex.
Risks	The main risks are that the final monthly cost may exceed the \$350 planning target, the managed platform may become limiting as the product grows, and some operational ownership is still not fully confirmed.
Requirements	R01, R02, R03, R04
Reason	Not recorded.
Status	accepted

11 / COMMUNICATION

Communicating the decision

Engineer	The Python app will run on a managed application platform with PostgreSQL as a managed database. HTTPS protects traffic, the database is encrypted and backed up, and monitoring collects logs and metrics. This reduces the amount of infrastructure the team must manage.
Cto	This option supports the four-week launch and reduces operational work for the two-engineer team. It is a good fit for the current standard application and modest traffic. The main limitation is less infrastructure control if the product becomes more complex later.
Cfo	The goal is to stay close to the \$350 monthly planning target while also reducing engineering time spent on infrastructure. We still need to validate the full monthly cost, including backups, monitoring, data transfer, resilience, and operations.
Ceo	This approach helps the team launch the customer portal quickly without spending most of the sprint on infrastructure. It keeps customer data protected and lets the engineers focus more on product features and customers.
Skip to content	

Analogy

It is like renting a managed office instead of building your own building. You have less control over every detail, but you can move in faster and spend less time on maintenance.

Without Service Names

The customer connects securely to a managed web application platform. The application uses a managed database with encryption and automatic backups, a managed email service, scheduled background work, and centralized monitoring. This keeps the architecture simple for a small team while supporting the launch and security requirements.

12 / REVIEW

Review and responses

Reviewer: Mock · local simulation

Decision-critical evidence · critical

A decision-critical requirements constraint was not established. Discover it, or explicitly defer the decision with a measurable validation plan.

Response: accept

Rationale: I agree. We still do not know if the application needs custom networking, operating system access, or another special requirement. This could change the platform choice.

Decision-critical evidence · critical

A decision-critical operations constraint was not established. Discover it, or explicitly defer the decision with a measurable validation plan.

Response: accept

Rationale: I agree. We know the team has only two engineers, but we do not know how much time they can spend on deployment, patching, incidents, and recovery. This could change the decision.

[Skip to content](#)

Facts versus assumptions · warning

Your notes do not distinguish both assumptions and unknowns. Identify what is inferred, what remains unmeasured, and which uncertainty could change this decision.

Response: accept

Rationale: I agree. The four-week launch, two engineers, modest traffic, and \$350 target are facts. The absence of special infrastructure needs and the final monthly cost are still assumptions or unknowns.

Accepted risk · warning

You marked R03, R04 at risk. Explain the mitigation, owner, and threshold that would trigger an alternative.

Response: accept

Rationale: I agree. The budget owner should review the full monthly cost before production. If the solution stays more than 15% above the \$350 target, we should review a simpler or cheaper option.

Validation · warning

No evidence request supports the recommendation yet. Defend why the available client statements are enough, or identify one measurement that could disprove your diagnosis.

Response: accept

Rationale: I agree. We still need evidence. A full monthly cost estimate and a deployment test could prove that the managed platform is not the right fit.

[Skip to content](#)

Trade-offs · info

You recommend “Deploy the Python web application on Elastic Beanstalk and use RDS PostgreSQL for the database. Use managed services for email, backups, monitoring, and encryption”. Defend it against your least complex alternative using a specific requirement and the team’s operating capacity. A more sophisticated service is not automatically a better fit.

Response: partial

Rationale: I partially agree. A single server would be simpler, but it creates more maintenance and resilience risk for a two-engineer team. The managed platform adds some cost but better supports the four-week launch.

Change of mind · info

Your first reconsideration threshold is “Monthly cost stays well above the \$350 target, or the team needs custom networking, operating system access, or platform features that the managed option cannot support.”. Explain who will observe this signal and when the decision will be reviewed.

Response: accept

Rationale: I agree. The engineering team and budget owner should review cost and technical requirements before production. If the platform becomes too expensive or lacks needed control, we should reconsider ECS Fargate.

Communication · All audiences · info

Your four explanations use different perspectives. Verify that their promised cost, risk, and delivery outcomes remain consistent with the same architecture decision.

Response: accept

Rationale: I agree. All explanations should stay consistent: the same architecture reduces operational work and supports a fast launch, but it still has cost and flexibility limits.

[Skip to content](#)

Final decision and lessons

Final Decision	I keep the managed platform recommendation because it still best supports the four-week launch and reduces infrastructure work for the two-engineer team. After review, I would not approve production until we confirm special infrastructure requirements, operational capacity, and the full monthly cost. If the platform needs more control or stays too far above the \$350 target, I would review the decision.
Lessons	I learned that architecture decisions depend not only on business and technical requirements, but also on the team’s capacity and limits. A good solution must fit what the team can realistically build, operate, and support.

EVALUATION

Evaluation

90/100

Needs revision

Transparent rubric · scenario rules

Critical misses

- A decision-critical requirements constraint was not established. Discover it, or explicitly defer the decision with a measurable validation plan.

Dimension	Score	Feedback
Discovery quality	11/20	3/6 discovery topics established; 0 reasoned evidence request(s). Keep assumptions separate from facts and pursue constraints that could change the decision.
Problem framing	10/10	A strong framing separates the client’s symptoms from an actionable business problem and makes uncertainties explicit.
Architecture	15/15	2 alternatives compared; 100% of requirements traced. The rubric rewards documented fit, not particular cloud services.
Skip to content		
Trade-off analysis	20/20	Trade-offs need explicit advantages, accepted costs, simpler alternatives, and connections to requirements. Numeric ratings do not determine this score.

Dimension	Score	Feedback
Cost and business reasoning	9/10	Tie costs to a stated budget, observed baseline, or financial impact. Ranges and explicitly labeled estimates are acceptable.
Security, reliability, operations	10/10	Document failure modes, security controls, operational ownership, and a practical validation or exit path.
Communication	10/10	Adapt the same decision to each audience. Executive explanations should describe business outcomes and risk in plain language.
Limits and change of mind	5/5	Revisit the decision when an observable threshold changes. A POC should have measurable success and stop criteria.

REASONING HISTORY

How the work changed

2 immutable snapshot(s) are saved.

First analysis

Final revision

Field	Before review	Final revision
Final Decision	Not recorded	I keep the managed platform recommendation because it still best supports the four-week launch and reduces infrastructure work for the two-engineer team. After review, I would not approve production until we confirm special infrastructure requirements, operational capacity, and the full monthly cost. If the platform needs more control or stays too far above the \$350 target, I would review the decision.
Lessons	Not recorded	I learned that architecture decisions depend not only on business and technical requirements, but also on the team's capacity and limits. A good solution must fit what the team can realistically build, operate, and support.

PORTFOLIO

Publication status

Published in the portfolio.

This PDF reflects the saved case.